

CampusVoice

AI-Powered Student Sentiment Agent

Step-by-Step Logic Flow & Architecture

CampusVoice answers plain-English questions about professors, courses, and campus life using real student review data from Rate My Professors — powered by Google Gemini and Elasticsearch.

Gemini 2.5 Flash

Elasticsearch

Elastic MCP Server

Flask

Cloud Run

Python 3.11



Google Cloud Rapid Agent Hackathon — June 2026

Partner Track: Elastic | Category: Education & Research

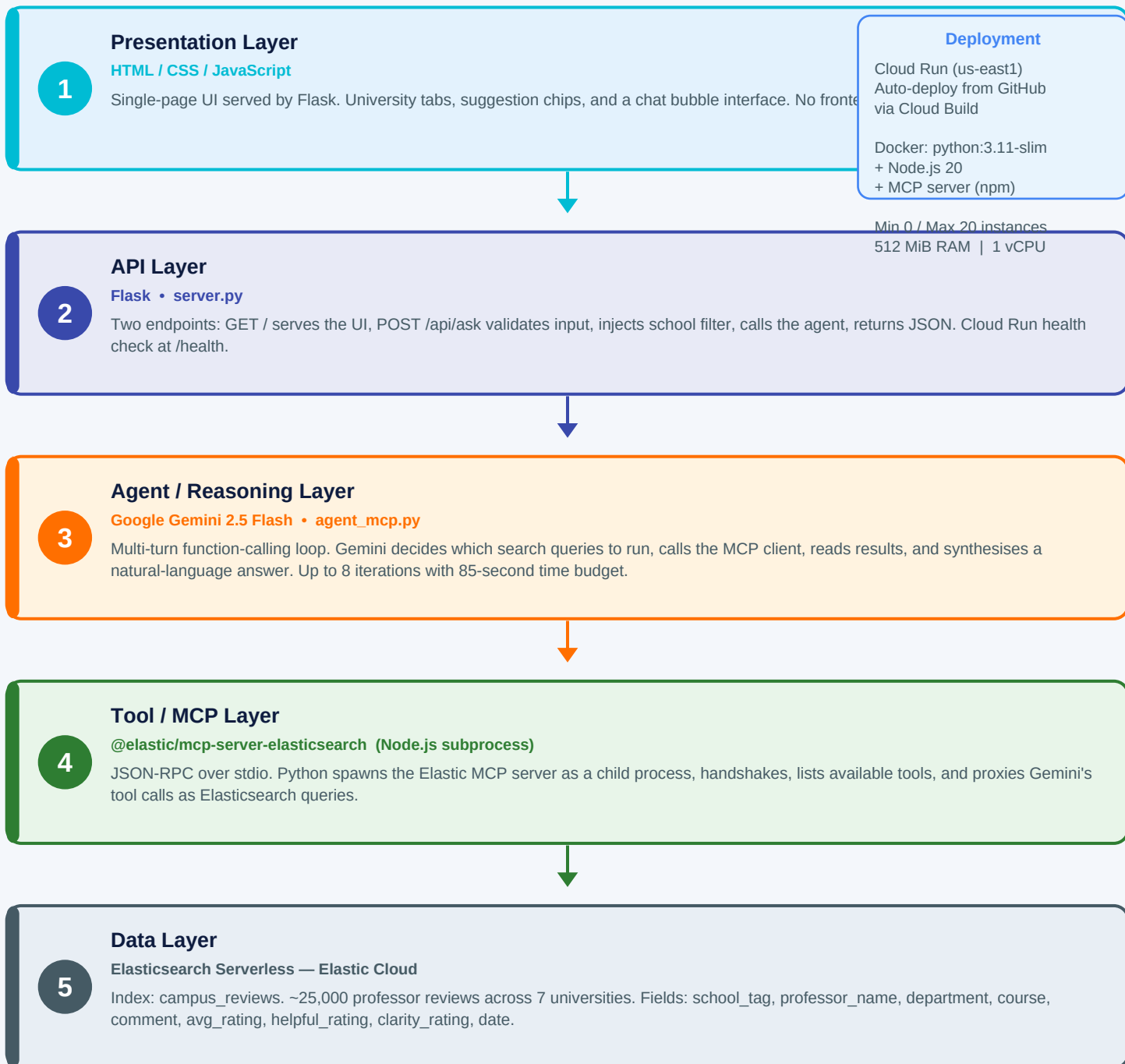
Live: <https://campus-voice-agent-420887396772.us-east1.run.app>

GitHub: <https://github.com/srivi19/campus-voice-agent>

System Architecture

How all the pieces fit together

CampusVoice is built from five cooperating layers. Each handles one concern.



Step-by-Step Request Flow

Tracing a single question: "Which professors are most praised at Vanderbilt?"

1 User Types a Question

Browser UI

User selects "Vanderbilt" tab and types: "Which professors are most praised?" Browser sends POST /api/ask with { "question": "...", "school": "Vanderbilt University" }

```
POST /api/ask
{ "question": "Which professors are most praised?",
  "school": "Vanderbilt University" }
```

2 Flask Validates & Enriches

server.py

Input is validated (length < 500 chars, non-empty). School filter is prepended to the question so the agent knows to restrict results.

```
full_question = "[Filter to Vanderbilt University only]
Which professors are most praised?"
```

3 Agent Starts MCP Client

agent_mcp.py

MCPClient spawns @elastic/mcp-server-elasticsearch as a subprocess. JSON-RPC handshake (initialize → notifications/initialized). Tools listed: elasticsearch_search, elasticsearch_get.

```
subprocess.Popen(['mcp-server-elasticsearch'])
→ initialize handshake (30s timeout)
→ tools/list returns available tools
```

4 Gemini Decides the Search

Gemini 2.5 Flash

Gemini receives the question + system prompt + tool definitions. It reasons: to find most praised professors, search campus_reviews filtered by school_tag='vanderbilt', sorted by avg_rating descending, size 50.

```
Function call: elasticsearch_search
{ index: 'campus_reviews',
  query: { term: { school_tag: 'vanderbilt' } },
```

5 MCP Executes Elasticsearch

Elastic MCP Server + Elasticsearch

MCPClient forwards the function call as JSON-RPC tools/call to the subprocess. The MCP server translates it to an Elasticsearch DSL query and executes it against Elastic Cloud.

```
tools/call → ES query → Elastic Cloud
Returns: top 50 reviews with professor
names, ratings, and comments
```

6 Gemini Synthesises the Answer

Gemini 2.5 Flash

Gemini reads the search results, identifies top-rated professors, quotes representative reviews, and writes a structured summary. If it needs more data, it loops back to step 4 (up to 8 iterations).

```
Response: 'Top praised professors at Vanderbilt:
1. Prof. X (avg 4.9) - students call her...
2. Prof. Y (avg 4.8) - known for...'
```

7 Flask Returns JSON

server.py

The synthesised answer is returned from agent_mcp.ask(). Flask wraps it as JSON and sends HTTP 200. On timeout (>85s) a graceful error is returned instead.

```
{ "answer": "Top praised professors at
Vanderbilt: 1. Prof. X..." }
```

8 Chat Bubble Renders Answer

Browser UI

JavaScript receives the JSON response and appends a styled chat bubble to the conversation. The answer appears within 5-15 seconds of the user pressing Ask.

```
Chat bubble with formatted answer
appears below the user's question
```

■ Gemini may loop steps 4-5 up to 8× before synthesising

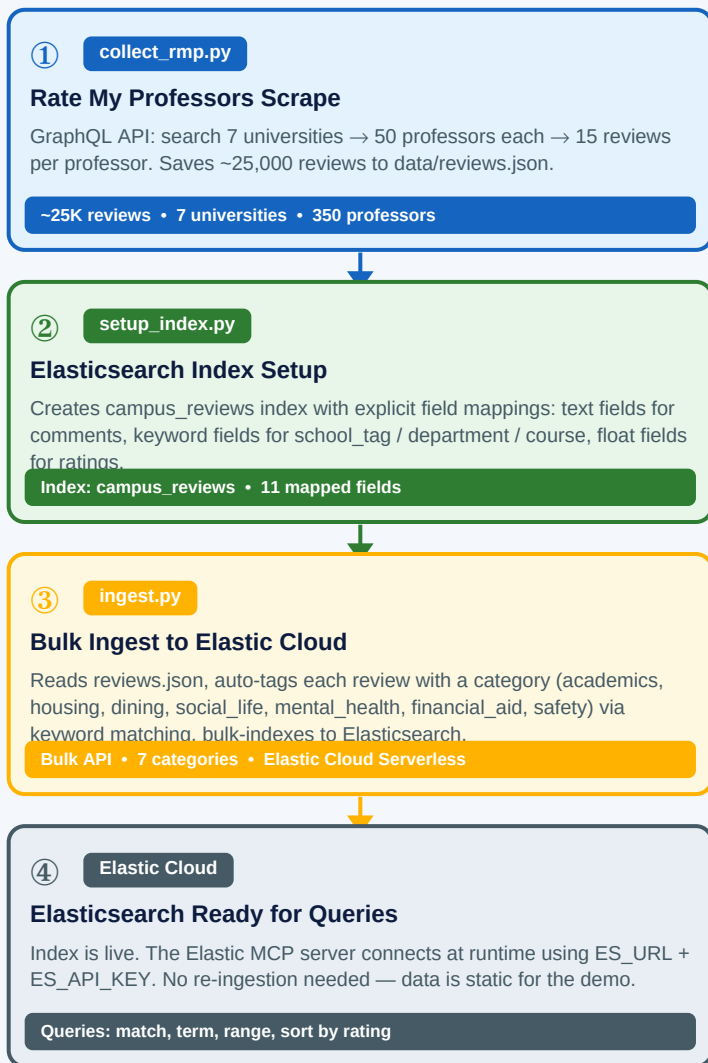
Agent Loop & Data Pipeline

Inside the reasoning loop and how the data was collected

Agent Reasoning Loop (agent_mcp.py)



Data Pipeline (one-time setup)



7

Universities

~25K

Student Reviews

8

Agent Iterations Max

85s

Time Budget

5

503 Retries

2

MCP Tools Available